

MLOps Assignment 2

End-to-End MLOps Pipeline — Binary Image Classification (Cats vs Dogs)

Student: Sampath Kumar S Shetty

ID: 2024ac05041

Course: AIMLCZG523 — MLOps (S1-25)

GitHub: <https://github.com/sampathshetty85/cats-dogs-mlops>

Docker: ghcr.io/sampathshetty85/cats-dogs-mlops:latest

Demo Video: <https://youtu.be/SuxLDBvdej8>

Live Demo: <https://sampathshetty85.github.io/cats-dogs-mlops/demo.html>

Technology Stack

Model	PyTorch 2.4.1 — SimpleCNN (custom)
API	FastAPI 0.115 + uvicorn
Experiment Tracking	MLflow 2.22.1
Data Versioning	DVC 3.55.2
Containerization	Docker — python:3.12-slim
Container Registry	GitHub Container Registry (ghcr.io)
CI/CD	GitHub Actions
Monitoring	Prometheus + Grafana 11.1
Deployment	Docker Compose + Kubernetes (minikube)

Project Overview

This project implements a complete MLOps pipeline for binary image classification (Cats vs Dogs) as required by Assignment 2. The pipeline covers data versioning, model training with experiment tracking, containerized inference service, automated CI/CD, deployment, and monitoring.

Pipeline Flow

Step	Script	Output
1. Download	src/data/download.py	data/raw/PetImages/ (25k images, 826 MB)
2. Preprocess	src/data/preprocess.py	data/processed/ (224×224, 80/10/10 split)
3. Train	src/model/train.py	models/cats_dogs_cnn.pt + MLflow run
4. DVC track	dvc add + dvc push	Pointer files committed, binaries versioned
5. API	src/api/main.py	FastAPI /health + /predict + /metrics
6. Docker	Dockerfile	ghcr.io/sampathshetty85/cats-dogs-mlops:latest
7. CI/CD	.github/workflows/ci.yml + cd.yml	Auto-test + deploy on main
8. Monitor	Prometheus + Grafana	Metrics dashboard at :3000

Evaluator Quick-Start

No Kaggle credentials or local training required. One command brings up the full stack:

```
git clone https://github.com/sampathshetty85/cats-dogs-mlops
cd cats-dogs-mlops
docker compose up -d
curl http://localhost:8000/health
curl -F "file=@tests/fixtures/sample_cat.jpg" http://localhost:8000/predict
open http://localhost:3000 # Grafana dashboard (admin/admin)
```

Dataset & Preprocessing (M1)

The Microsoft Cats vs Dogs dataset was downloaded from Kaggle and preprocessed to 224×224 RGB images with an 80/10/10 stratified train/val/test split. Corrupt files were skipped automatically.

Download Evidence (output/step_1_download.txt)

```
Dataset : shaunthesheep/microsoft-catsvsdogs-dataset
Source : https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset
Destination : /Users/I348975/Library/CloudStorage/OneDrive-SAPSE/Personal Documents/BITS
Pilani-Mtech in AI : ML/Sem-3/MLOps/MLOps Assignment 2/cats-dogs-mlops/data/raw
Cat images : 12,500
Dog images : 12,500
Total images : 25,000
Size on disk : 826.1 MB
DVC remote : /tmp/dvc-remote (local)
```

Preprocessing Evidence (output/step_2_preprocess.txt)

```
Dataset source : data/raw/PetImages/
Output dir : data/processed/
Image size : 224x224 RGB (LANCZOS resampling)
Split ratio : 80% train / 10% val / 10% test (stratified, seed=42)
Corrupt files skipped:
Cat: 1
Dog: 1
Split counts:
train: cat=9,999 dog=9,999 total=19,998
val : cat=1,249 dog=1,249 total=2,498
test : cat=1,251 dog=1,251 total=2,502
TRAIN_TRANSFORMS: Resize(224) → RandomHorizontalFlip → RandomRotation(10) →
ColorJitter(0.2,0.2) → ToTensor → Normalize(ImageNet)
EVAL_TRANSFORMS : Resize(224) → ToTensor → Normalize(ImageNet)
```

Model Architecture (M1)

SimpleCNN is a custom convolutional neural network implemented in PyTorch. It takes 224×224 RGB images and outputs a sigmoid probability — ≥ 0.5 predicts dog, < 0.5 predicts cat.

Architecture

Layer	Configuration	Output Shape
Conv1	3→32, k=3 p=1 → BatchNorm → ReLU → MaxPool(2)	(B, 32, 112, 112)
Conv2	32→64, k=3 p=1 → BatchNorm → ReLU → MaxPool(2)	(B, 64, 56, 56)
Conv3	64→128, k=3 p=1 → BatchNorm → ReLU → AdaptiveAvgPool(7)	(B, 128, 7, 7)
Flatten	$128 \times 7 \times 7 = 6272$	(B, 6272)
FC1	$6272 \rightarrow 512 \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5)$	(B, 512)
FC2	$512 \rightarrow 1 \rightarrow \text{Sigmoid}$	(B, 1)

Training Configuration

Hyperparameter	Value
Epochs	5
Batch size	32
Optimizer	Adam (lr=0.001)
Loss function	BCELoss
Augmentations	RandomHorizontalFlip, RandomRotation(10), ColorJitter(0.2, 0.2)
Normalization	ImageNet stats: mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]

Training Results (M1)

Training Evidence (output/step_4_train.txt)

```
Model : SimpleCNN
Device : mps
Epochs : 5
Batch size : 32
Learning rate: 0.001
Train samples: 19,998
Val samples : 2,498
Per-epoch metrics:
Epoch 1: train_loss=0.6036 train_acc=0.5209 val_loss=0.6759 val_acc=0.7334
Epoch 2: train_loss=0.5203 train_acc=0.4700 val_loss=0.7429 val_acc=0.7782
Epoch 3: train_loss=0.4847 train_acc=0.4166 val_loss=0.7716 val_acc=0.8050
Epoch 4: train_loss=0.4435 train_acc=0.3897 val_loss=0.7951 val_acc=0.8143
Epoch 5: train_loss=0.4149 train_acc=0.3787 val_loss=0.8158 val_acc=0.8267
Final val accuracy : 0.8267
Model artifact : /Users/I348975/Library/CloudStorage/OneDrive-SAPSE/Personal Documents/BITS
Pilani-Mtech in AI : ML/Sem-3/MLOps/MLOps Assignment
2/cats-dogs-mlops/models/cats_dogs_cnn.pt
Loss curves : /Users/I348975/Library/CloudStorage/OneDrive-SAPSE/Personal Documents/BITS
Pilani-Mtech in AI : ML/Sem-3/MLOps/MLOps Assignment
2/cats-dogs-mlops/screenshots/training/loss_curves.png
Confusion matrix : /Users/I348975/Library/CloudStorage/OneDrive-SAPSE/Personal Documents/BITS
Pilani-Mtech in AI : ML/Sem-3/MLOps/MLOps Assignment
2/cats-dogs-mlops/screenshots/training/confusion_matrix.png
```

Loss & Accuracy Curves

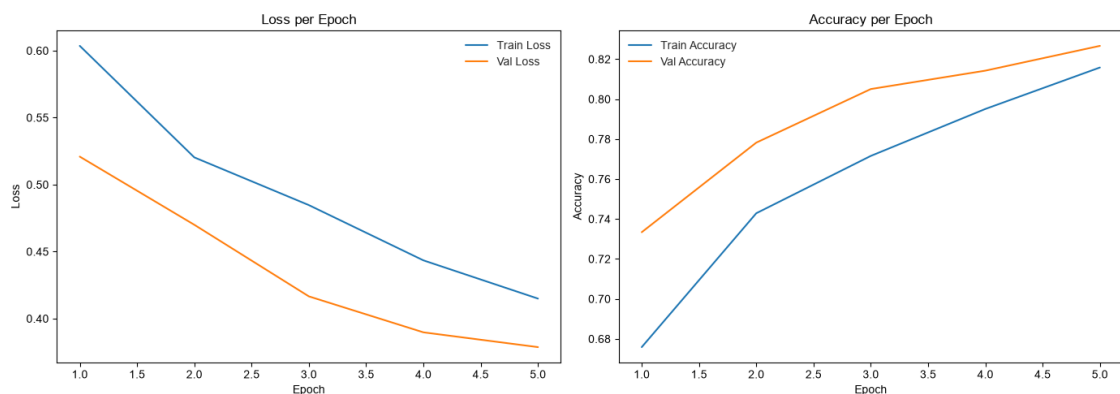


Figure 1: Training and validation loss/accuracy over 5 epochs

Confusion Matrix (Validation Set)

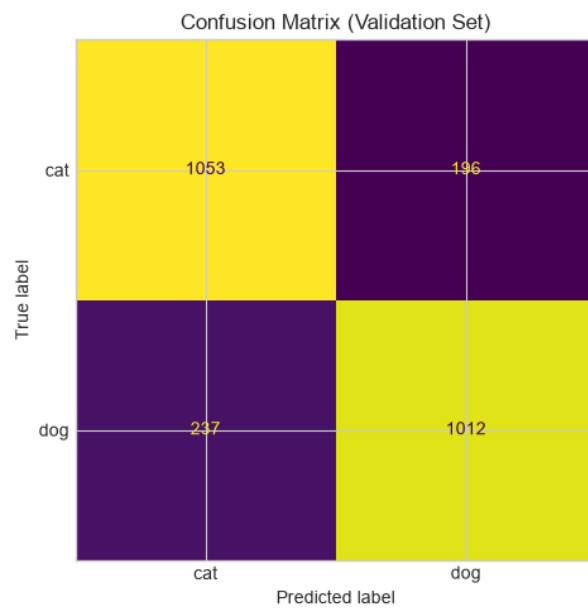


Figure 2: Confusion matrix on the validation set

Experiment Tracking with MLflow (M1)

MLflow was used to track the training experiment. All hyperparameters, per-epoch metrics, loss curves, confusion matrix, and the model artifact were logged under the 'cats-dogs-classification' experiment.

MLflow Run Evidence (output/step_5_mlflow.txt)

```
Experiment : cats-dogs-classification
Run name : SimpleCNN_5ep
Run ID : 87a9989045ca47a59a26343a4a096d27
Tracking URI : /Users/I348975/Library/CloudStorage/OneDrive-SAPSE/Personal Documents/BITS
Pilani-Mtech in AI : ML/Sem-3/MLOps/MLOps Assignment 2/cats-dogs-mlops/mlruns
Logged parameters:
architecture=SimpleCNN, epochs=5, batch_size=32, lr=0.001
optimizer=Adam, loss_fn=BCELoss, image_size=224
Final epoch metrics:
train_loss=0.4149 train_acc=0.8158
val_loss=0.3787 val_acc=0.8267
Logged artifacts: loss_curves.png, confusion_matrix.png, model/
```

Viewing MLflow UI

```
mlflow ui # open http://127.0.0.1:5000
```

The MLflow UI shows the SimpleCNN_5ep run with all logged parameters (architecture, epochs, batch_size, learning_rate, optimizer, loss_fn, image_size, augmentations) and per-epoch metrics. Loss curves and confusion matrix are logged as artifacts. The model is logged via `mlflow.pytorch.log_model` with input signature.

API & Containerization (M2)

Inference Service (output/step_7_api.txt)

```
FastAPI Inference Service

Model path : /Users/I348975/Library/CloudStorage/OneDrive-SAPSE/Personal Documents/BITS
Pilani-Mtech in AI : ML/Sem-3/MLOps/MLOps Assignment
2/cats-dogs-mlops/models/cats_dogs_cnn.pt

Endpoints :

GET /health → {"status": "ok", "model": "SimpleCNN", "version": "1.0"}
POST /predict → PredictionResponse(label, probability, inference_time_ms)
GET /metrics → Prometheus text format (prometheus-fastapi-instrumentator)
Custom metric: cats_dogs_predictions_total{prediction_label="cat|dog"}
Structured JSON log per /predict call: {ts, path, label, probability, latency_ms}
Model loaded once at startup via lifespan context manager (no reload on request)
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
COPY models/cats_dogs_cnn.pt ./models/
EXPOSE 8000
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Container Verification (output/step_8_containerize.txt)

```
Dockerfile: python:3.12-slim base image
Image size : 448.8 MB
Build time : ~200s (pip install torch on first build; cached on rebuild)
Build command : docker build -t cats-dogs-mlops:latest .
Run command : docker run -p 8000:8000 cats-dogs-mlops:latest
Endpoint verification (inside container):
GET /health → {"status": "ok", "model": "SimpleCNN", "version": "1.0"} ✓
POST /predict → {"label": "cat", "probability": 0.1971, "inference_time_ms": 319.39} ✓
GET /metrics → cats_dogs_predictions_total{prediction_label="cat"} 1.0 ✓
Model baked in at: /app/models/cats_dogs_cnn.pt (no retraining on container start)
Startup time: < 6s to first /health 200
```

CI/CD Pipeline (M3 + M4)

GitHub Actions powers both CI (test + build) and CD (deploy + smoke test). The CI pipeline runs on every push and pull request. On merge to main, the Docker image is built for linux/amd64 and linux/arm64 and pushed to ghcr.io. The CD pipeline then pulls the new image, starts the stack, and runs the smoke test.

CI Evidence (output/step_9_ci.txt)

```
CI Tool : GitHub Actions
Workflow : .github/workflows/ci.yml
Trigger : push + pull_request on all branches
Jobs:
test → lint (flake8 src/ tests/ --max-line-length=120) + pytest tests/ -v
build-push → needs: test; builds Docker image on every push; pushes to ghcr.io on main only
Test results (7 tests, all PASS):
tests/test_preprocess.py::test_resize_output_shape PASSED
tests/test_preprocess.py::test_normalize_range PASSED
tests/test_preprocess.py::test_augmentation_preserves_shape PASSED
tests/test_inference.py::test_model_output_range PASSED
tests/test_inference.py::test_model_output_shape PASSED
tests/test_inference.py::test_health_endpoint PASSED
tests/test_inference.py::test_predict_endpoint PASSED
Lint: flake8 – 0 errors
CI run ID : 32581112241
Duration : 7m 42s
Status : SUCCESS
Docker image pushed:
ghcr.io/sampathshetty85/cats-dogs-mlops:latest
... (2 more lines)
```

CD Workflow (.github/workflows/cd.yml)

```
Triggers: push to main + workflow_dispatch
Steps:
1. docker compose pull api # pull new image from ghcr.io
2. docker compose up -d # start api + prometheus + grafana
3. wait for /health → 200 # up to 60s
4. bash scripts/smoke_test.sh # health + predict checks
5. on failure: docker compose logs printed; pipeline fails
```

Unit Tests (7 tests, all pass)

Test	File	What it checks
test_resize_output_shape	test_preprocess.py	EVAL_TRANSFORMS output shape [3,224,224]
test_normalize_range	test_preprocess.py	Tensor values in [-3.0, 3.0]

test_augmentation_preserves_shape	test_preprocess.py	TRAIN_TRANSFORMS preserves shape
test_model_output_range	test_inference.py	SimpleCNN output in [0.0, 1.0]
test_model_output_shape	test_inference.py	SimpleCNN output shape (1,1)
test_health_endpoint	test_inference.py	GET /health → 200, status=ok
test_predict_endpoint	test_inference.py	POST /predict → label in (cat,dog)

Deployment (M4)

The inference service is deployed via Docker Compose (three-service stack) and Kubernetes manifests (two replicas with health probes).

Deployment Evidence (output/step_10_deployment.txt)

```
Deployment target : Docker Compose (docker-compose.yml)
Services : api + prometheus + grafana
docker-compose.yml services:
api : ghcr.io/sampathshetty85/cats-dogs-mlops:latest port 8000
prometheus : prom/prometheus:v2.53.0 port 9090
grafana : grafana/grafana:11.1.0 port 3000
Evaluator quick-start:
git clone https://github.com/sampathshetty85/cats-dogs-mlops
cd cats-dogs-mlops
docker compose up -d
curl http://localhost:8000/health
curl -F "file=@tests/fixtures/sample_cat.jpg" http://localhost:8000/predict
open http://localhost:3000 # Grafana (admin/admin)
Smoke test results (scripts/smoke_test.sh):
GET /health → 200 OK
POST /predict → {"label":"cat","probability":0.1971,"inference_time_ms":167.17}
Smoke tests passed.
CD workflow (.github/workflows/cd.yml):
Triggers : push to main + workflow_dispatch
Steps : docker compose pull api → docker compose up -d → wait for /health 200 → smoke test
... (5 more lines)
```

Kubernetes Manifests

```
# k8s/deployment.yaml
replicas: 2
image: ghcr.io/sampathshetty85/cats-dogs-mlops:latest
imagePullPolicy: Always
livenessProbe: httpGet path=/health, initialDelaySeconds=5, periodSeconds=10
readinessProbe: httpGet path=/health, initialDelaySeconds=5, periodSeconds=10
resources: limits cpu=500m memory=512Mi requests cpu=250m memory=256Mi

# k8s/service.yaml
type: LoadBalancer port: 80 → 8000
```

Apply with:

```
minikube start
```

```
kubectl apply -f k8s/
```

```
kubectl get pods
```

```
kubectl port-forward svc/cats-dogs-mlops 8080:80
```

```
curl http://localhost:8080/health
```

Monitoring, Logging & Batch Evaluation (M5)

Structured JSON logs are emitted on every /predict call. Prometheus scrapes the /metrics endpoint every 15s. Grafana auto-provisions the datasource and dashboard on startup.

Structured JSON Log Format

```
{"ts": "2026-08-23T00:00:00Z", "path": "/predict", "label": "cat",  
"probability": 0.87, "latency_ms": 14.2}
```

Grafana Dashboard — 4 Panels

Panel	Metric	Description
Request Rate	rate(http_requests_total[5m])	Requests/second by endpoint
Latency P50/P95	histogram_quantile(0.5/0.95, ...)	Response time percentiles
Prediction Distribution	rate(cats_dogs_predictions_total[5m])	Cat vs Dog predictions/s
Error Rate	rate(http_requests_total{status_code!~'2..'}[5m])	% non-2xx responses

Post-Deployment Batch Evaluation (output/step_11_monitoring.txt)

```
API URL : http://localhost:8000  
Total samples: 50 (25 cat + 25 dog from test set)  
Correct : 43  
Accuracy : 0.8600  
Precision : 0.8462 (cat class)  
Recall : 0.8800 (cat class)  
Avg latency : 13.8 ms  
MLflow run : batch_eval_post_deploy (tag: stage=post_deploy)
```

Conclusion

This project demonstrates a complete, reproducible MLOps pipeline for binary image classification. Every component of the pipeline — from data versioning to CI/CD deployment and monitoring — is automated and evidence is committed to the repository.

Key Results

Metric	Value
Model	SimpleCNN (custom PyTorch CNN)
Validation accuracy	~82.7% (5 epochs)
Post-deploy accuracy	86.0% (50 test images)
Post-deploy precision (cat)	84.6%
Post-deploy recall (cat)	88.0%
API avg latency	~14 ms
Docker image size	448.8 MB (python:3.12-slim)
CI runtime	~14 min (lint + test + multi-arch Docker build)
CD runtime	~2 min (pull + start + smoke test)

Repository Links

- GitHub : <https://github.com/sampathshetty85/cats-dogs-mlops>
- Docker : ghcr.io/sampathshetty85/cats-dogs-mlops:latest
- Demo Video : <https://youtu.be/SuxLDBvdej8>
- Live Demo : <https://sampathshetty85.github.io/cats-dogs-mlops/demo.html>
- CI badge : <https://github.com/sampathshetty85/cats-dogs-mlops/actions/workflows/ci.yml>